

Checkpoint

Introduction

remembers the context

- The checkpoint design pattern is a durability mechanism used in agentic AI systems
- It saves the complete execution state of an agent or workflow at defined points during execution
- Each saved snapshot is called a checkpoint
- A checkpoint allows an agent to resume from an exact position after a crash, interruption, or deliberate pause
- It transforms an agent from a fragile, single-process script into a resilient, long-running service
- The pattern is analogous to save points in video games or database transaction logs in distributed systems

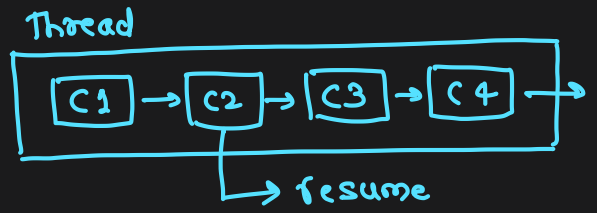
The Problem It Solves

- Agentic workflows are often long-running and multi-step
- They involve expensive LLM calls, external tool invocations, and complex branching logic
- A failure at step 99 of a 100-step process, without checkpointing, forces a complete restart
- This wastes time, tokens, money, and API rate limits
- Human-in-the-loop workflows require the ability to pause indefinitely and resume later
- Multi-turn conversations require persistent memory across separate process invocations
- Checkpointing addresses all of these requirements with a single architectural mechanism
  - ↳ persists state in checkpoint file

Core Concepts and Terminology

→ can be saved in the form of SQLite DB, json file, postgreSQL

- **Checkpoint:** A serialized snapshot of execution state at a specific point in time
- **Thread:** A logical conversation or workflow instance identified by a unique ID, used to group related checkpoints
- **Superstep:** A batch of execution in which multiple nodes or executors run before the next checkpoint is taken
- **Checkpointer:** The component responsible for saving and loading checkpoints
- **RunState:** An object representing the full, serializable state of an in-progress run
- **Interrupt:** A deliberate pause in execution that surfaces a request for external input → HITL → approve / reject
- **Resume:** The act of reconstructing state from a checkpoint and continuing execution



What Gets Captured in a Checkpoint

- System → Tool Call
- Human
- AI

- **Conversation State:** The full message history that reconstructs the agent's context window
- **Execution State:** The current iteration count, execution phase, and pending node or tool calls
- **Executor States:** Custom state maintained by individual agents, nodes, or workflow executors
- **Shared State:** Variables and data shared across the entire workflow or graph
- **Pending Messages:** Messages queued for delivery in the next superstep
- **In-Flight Requests:** Outstanding requests awaiting responses, such as human approvals
- **Metadata:** Step numbers, timestamps, source of the checkpoint, and version information
-



When Checkpoints Are Created

- **Superstep Boundaries:** After all executors in a workflow step complete their work
- **Phase Transitions:** Before an LLM call, after an LLM response, or after tool execution
- **Human Approval Points:** When the workflow pauses to wait for external input
- **Interval-Based:** Every N iterations or after processing a defined batch of items
- **Manual Triggers:** When the application explicitly requests a checkpoint
- **Entry Points:** At the start of a workflow to capture the original input for replayability
- Checkpoints are taken at deterministic boundaries to guarantee consistency across components

Resume and Recovery



- The saved checkpoint is loaded from storage into memory
- The agent or workflow context is reconstructed from the serialized state
- Pending nodes, tool calls, or approval requests are identified
- Execution continues from the exact point of interruption
- In some frameworks, message history is replayed into a new instance to rebuild context
- Stable identifiers are required to match saved state to the correct executors across process restarts
- Resume can occur in the same process, a new process, or even a different machine

Human-in-the-Loop Interruptions



- Checkpointing is the foundation for human-in-the-loop agent workflows
- A node or tool can explicitly request approval before performing a sensitive action
- The workflow pauses and saves a checkpoint at that exact moment
- The pending request is surfaced to a human reviewer through an API or UI
- The human provides a decision: approve, reject, or edit
- The workflow resumes from the checkpoint with the human's response injected
- Side effects before an interrupt must be idempotent, since the node may re-execute from the start on resume

Storage Backends and Durability



- **In-Memory Storage:** Fast and simple, suitable only for development and testing; loses state on restart
- **File-Based Storage:** Persists to disk, suitable for single-machine deployments and demos
- **SQLite Storage:** Lightweight, file-backed, and durable; good for small to medium projects
- **Relational Databases:** PostgreSQL, MySQL, and similar systems for production workloads
- **NoSQL Databases:** MongoDB, Redis, and similar systems for distributed or low-latency scenarios
- **Cloud Storage:** Managed services such as Cosmos DB for enterprise-grade durability
- Storage choice determines durability, scalability, and recovery guarantees

Best Practices

- Choose a persistent checkpointer for any production workload; never rely on in-memory storage
- Assign stable, unique identifiers to all executors and nodes to ensure reliable resume
- Design nodes so that side effects before interrupts are idempotent
- Place side effects after interrupts whenever possible, or isolate them in dedicated nodes
- Implement checkpoint pruning and retention policies to control storage costs
- Version your state schemas and executor definitions to maintain compatibility
- Test resume paths explicitly, including crash recovery and cross-process resumption
- Log checkpoint creation and resume events for observability and debugging